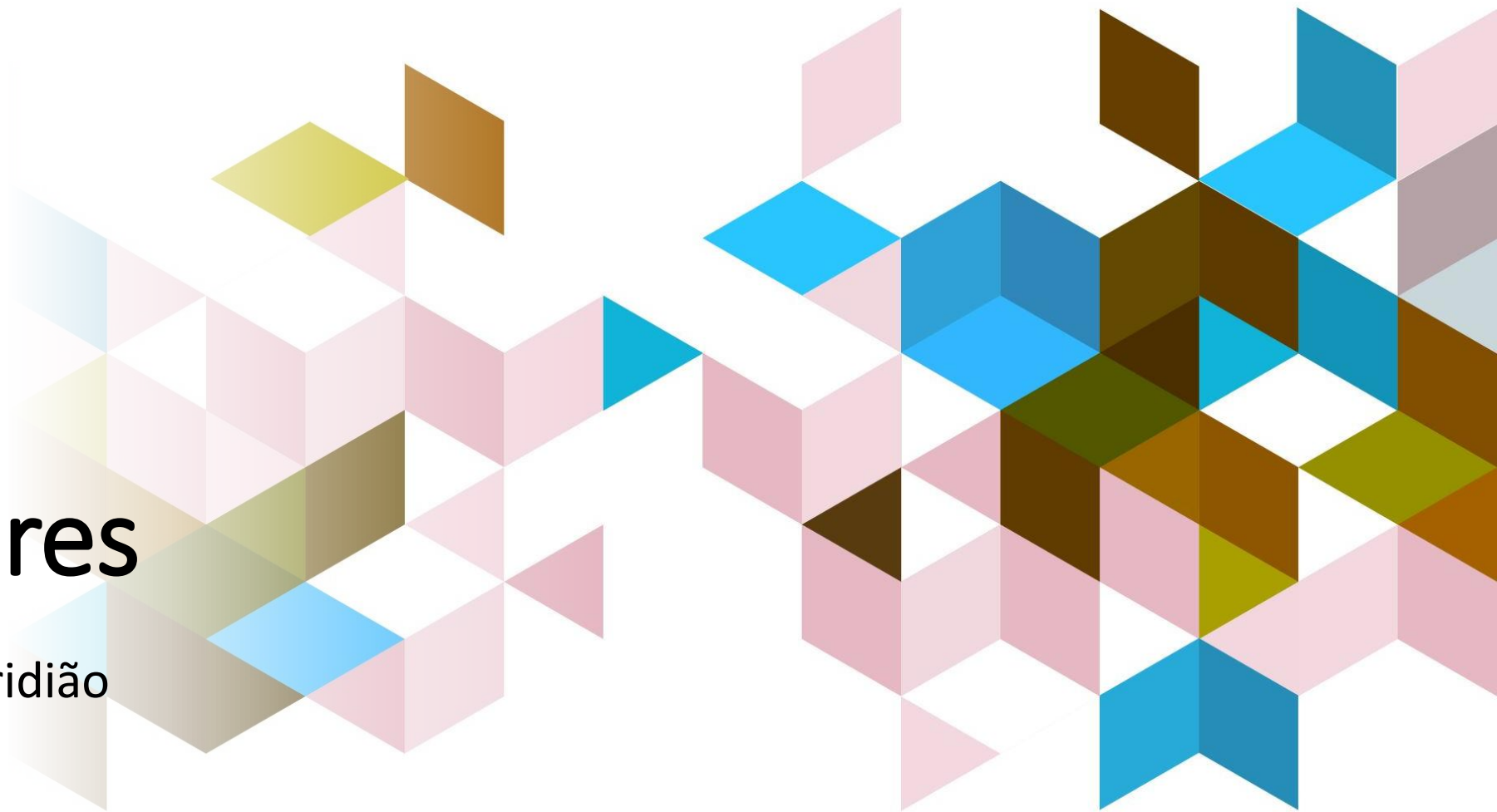


Construtores

Prof. Me. Hélio Esperidião



Método construtor

O método construtor determina que ações devem ser executadas quando é realizada criação de um objeto.

Em Java, o construtor é definido como um **método** cujo nome deve ser o **mesmo nome da classe** e sem indicação do tipo de retorno

O construtor é unicamente invocado no momento da criação do objeto através do operador new.

O retorno do operador new é uma referência para o objeto recém-criado. O construtor pode receber argumentos, como qualquer método. Usando o mecanismo de sobrecarga, mais de um construtor pode ser definido para uma classe. Veja por exemplo os construtores definidos para a classe Point de Java.

Construtor

Toda classe tem pelo menos um construtor sempre definido.

Se nenhum construtor não for explicitamente definido pelo programador da classe, um construtor padrão, que não recebe argumentos, é incluído para a classe pelo compilador Java.

No entanto, se o programador da classe criar pelo menos um método construtor, o construtor padrão **não será** criado automaticamente

Se o programador desejar mantê-lo, deverá criar um construtor sem argumentos explicitamente.

sequência de ações executada para a criação de um objeto:

01

O espaço para o objeto é alocado e seu conteúdo é inicializado.

02

O construtor da classe é invocado.

03

Os membros da classe são inicializados para o objeto, seguindo a ordem em que foram declarados na classe.

04

O restante do corpo do construtor é executado.

Construtor

É um método que possui o mesmo nome da classe.

Este método é executado antes de todos os outros quando a classe é instanciada.

Construtores



Construtores são utilizados basicamente para inicializar as variáveis e recursos básicos de uma classe.



O Java cria automaticamente o construtor padrão Classe(), que não recebe nenhum parâmetro.

Construtor é um método:

Obrigatório para instanciar a classe

Utilizado para inicializar as variáveis e recursos da classe

Pode chamar outros métodos

Não possui tipo de retorno

Aceita parâmetros

Permite sobrecarga (Polimorfismo por sobrecarga)

Construtor

```
public class Aluno {  
    private String nome;  
    private String curso;  
    Aluno(){  
        System.out.println("Eu sou o construtor");  
    }  
    public String getNome() {  
        return nome;  
    }  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
    public String getCurso() {  
        return curso;  
    }  
    public void setCurso(String curso) {  
        this.curso = curso;  
    }  
}
```

```
public class Teste {  
    public static void main(String[] args) {  
        Aluno a1 = new Aluno();  
    }  
}
```

Sobrecarga de construtores

```
class Aluno {  
    private String nome;  
    private String curso;  
    Aluno(){  
        this.nome = "sem nome";  
        this.curso = "sem curso";  
    }  
    Aluno(String nome, String curso){  
        this.nome=nome;  
        this.curso=curso;  
    }  
    Aluno(String nome){  
        this.nome=nome;  
        this.curso="sem curso";  
    }  
}
```

```
public class Teste {  
    public static void main(String[] args) {  
        Aluno a1 = new Aluno();  
        Aluno a2 = new Aluno("Hélio","Eng Computação");  
        Aluno a3 = new Aluno("Hélio");  
    }  
}
```

Sobrecarga de construtores

- Análise o código ao lado.
- Torne este código funcional.
- Implemente conceitos de polimorfismo e encapsulamento de dados.
- Programe a classe com a maior quantidade de polimorfismo e encapsulamento possível.

```
public class Ponto2D {  
    private double x;  
    private double y;  
    Ponto2D() {  
        System.out.println("Construtor vazio");  
    }  
    Ponto2D(double x, double y) {  
        System.out.println("Construtor double");  
    }  
    Ponto2D(int x, int y) {  
        System.out.println("Construtor int");  
    }  
    Ponto2D(float x, float y) {  
        System.out.println("Construtor float");  
    }  
  
    public double CalcularDistancia(Ponto2D p1, Ponto2D p2) {  
        return 0;  
    }  
  
    public double CalcularDistancia(double x1, double x2, double y1, double y2) {  
        return 0;  
    }  
  
    public static void main(String args[]) {  
        Ponto2D p = new Ponto2D();  
        p = new Ponto2D(1.1, 1);  
        p = new Ponto2D(1.2f, 1);  
    }  
}
```

O construtor obriga

- Veja que existe apenas um construtor na classe e ele exige que sejam passados dois parâmetros(nome e idade). Caso os parâmetros não sejam passados é gerado erro de compilação

```
public class Pessoa {  
    public int idade;  
    public String nome;  
  
    public void fazerAniversario(){  
        this.idade++;  
    }  
  
    Pessoa(String nome,int idade){  
        this.nome=nome;  
    }  
  
    public static void main(String args[]){  
        Pessoa p =new Pessoa();  
        System.out.println(p.nome);  
        System.out.println(p.idade);  
    }  
}
```



Funções do Garbage Collector

Alocar memória

Assegurar que quaisquer objetos referenciados permaneçam na memória;

Recuperar a memória alocada pelos objetos que não são mais alcançáveis pelas referências no código em execução.

Garbage Collection.

- O controle do Coletor de Lixo é feito pela JVM, que é quem decide quando ele será executado.
- Mas não existe garantia de quando o Garbage Collector será executado, ainda que seja possível ao programador solicitar via código Java que a JVM o execute.
 - `System.gc ();`
- Em geral, quando não há interferência do programador, a máquina virtual executa o Coletor de Lixo quando percebe que a memória está ficando sem espaço.

Tornando objetos explicitamente elegíveis à coleta de lixo

```
public class Teste {  
    public static void main(String[] args) {  
        Aluno a1 = new Aluno();  
        a1 = null;  
    }  
}
```

Tornando objetos explicitamente elegíveis à coleta de lixo

```
public class Teste {  
    public static void main(String[] args) {  
        Aluno a1 = new Aluno();  
        a1 = new Aluno();  
    }  
}
```

```
public class Teste {  
    public static void main(String[] args) {  
        Aluno a1 = new Aluno();  
        Aluno a2 = new Aluno();  
        a1 = a2;  
    }  
}
```

//obs: a1 e a2 apontam para a mesma posição de memória, portando a posição a1 não possui mais referência.

Destrutores

Não existem destrutores em java

Liberar recursos alocados por objetos é função do garbage collection.